
Decision Trees

INSTRUCTOR: MOAYAD ALNAMMI

malnammi.github.io

Notice

- Slides prepared by Moayad Alnammi – malnammi.github.io
- Slides that adapt figures or content from other sources have a link to the source.

This deck is published at malnammi.github.io under CC BY-NC-SA 4.0. Reuse and adaptation require attribution to the author and the sources above; commercial use is not permitted; adaptations must carry the same license.

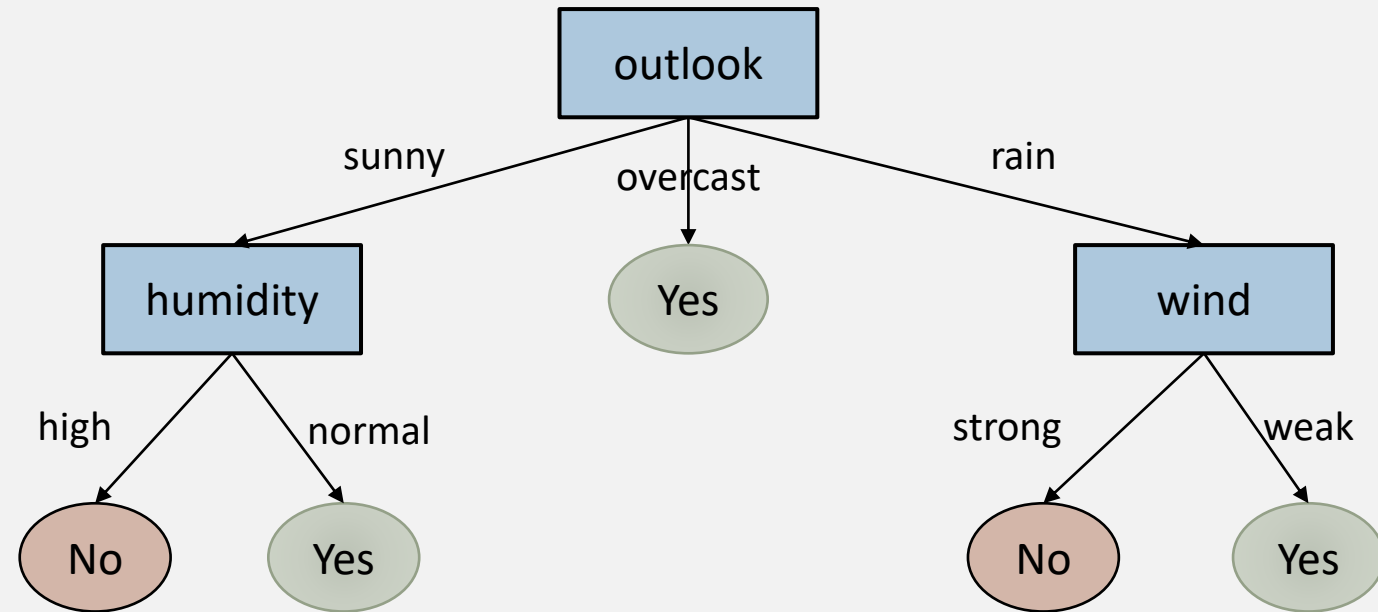
Outline

- Overview
- Decision trees with discrete features only (classification)
 - Gini Impurity of a set
 - Splitting on a feature and computing Gini Impurity of a split
 - Decision tree building using Gini Impurity
- Strengths and limitations of Decision Trees
- Scikit-learn's DT implementation

Decision Tree Example

- Consider this training set T where the decision to play depends on the given features outlook, temp, humidity, and wind.

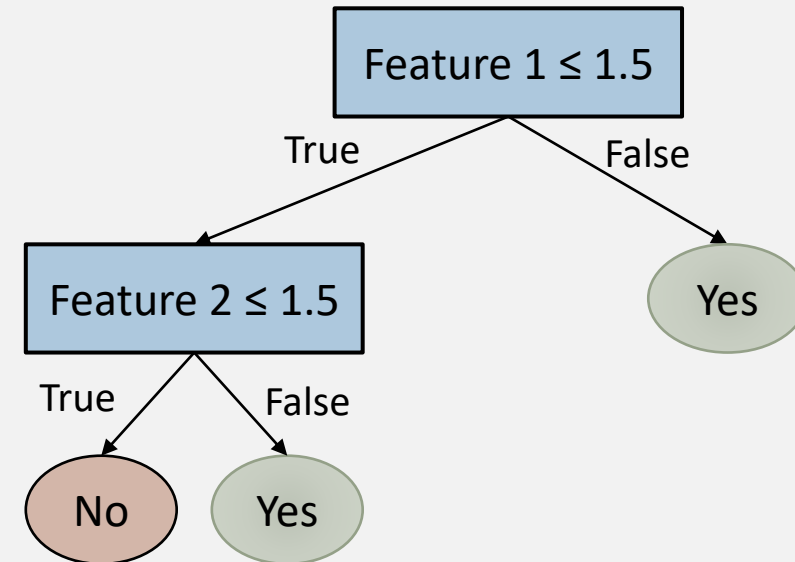
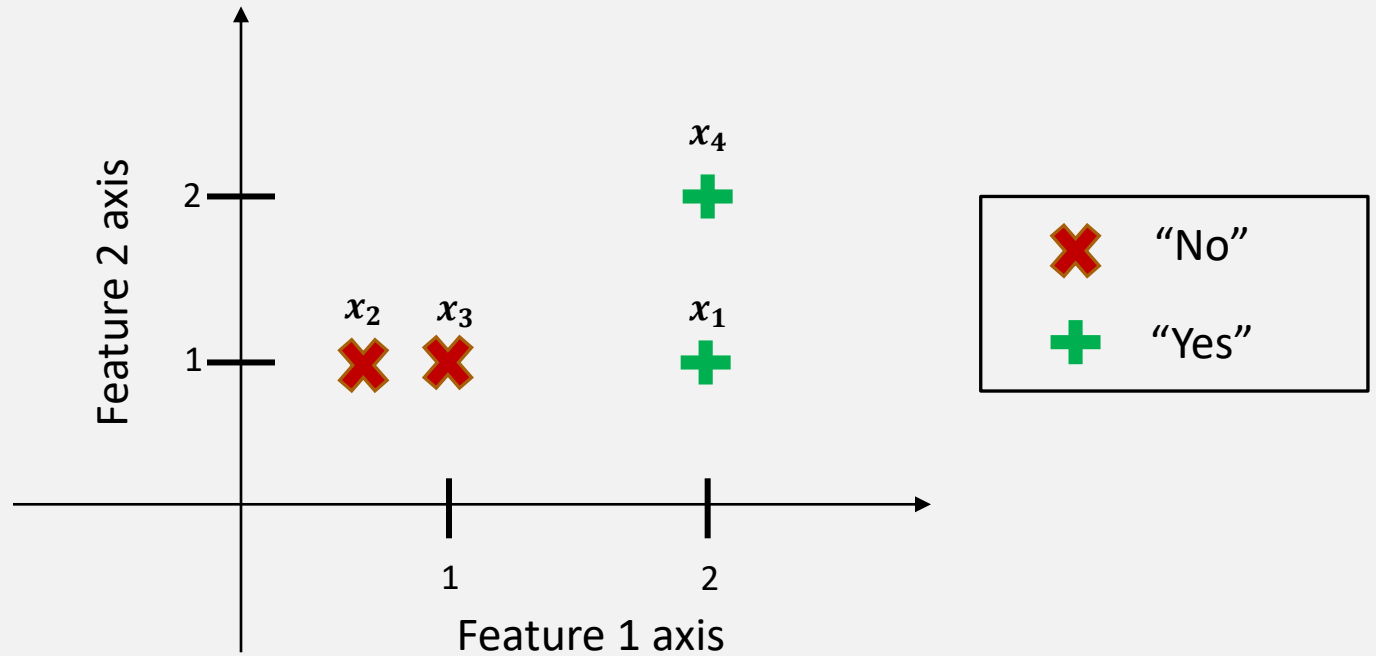
outlook	temp	humidity	wind	play
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No



- This decision tree fits the training data.
- Internal nodes** are decisions that **split** on a feature.
- Leaf nodes** are prediction decisions.
- How to predict on $x_{new} = [rain, cool, high, weak]$

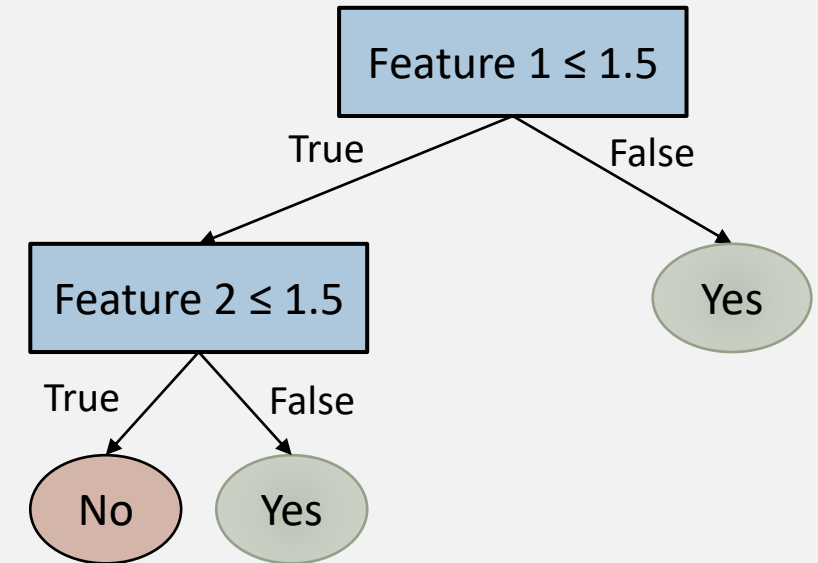
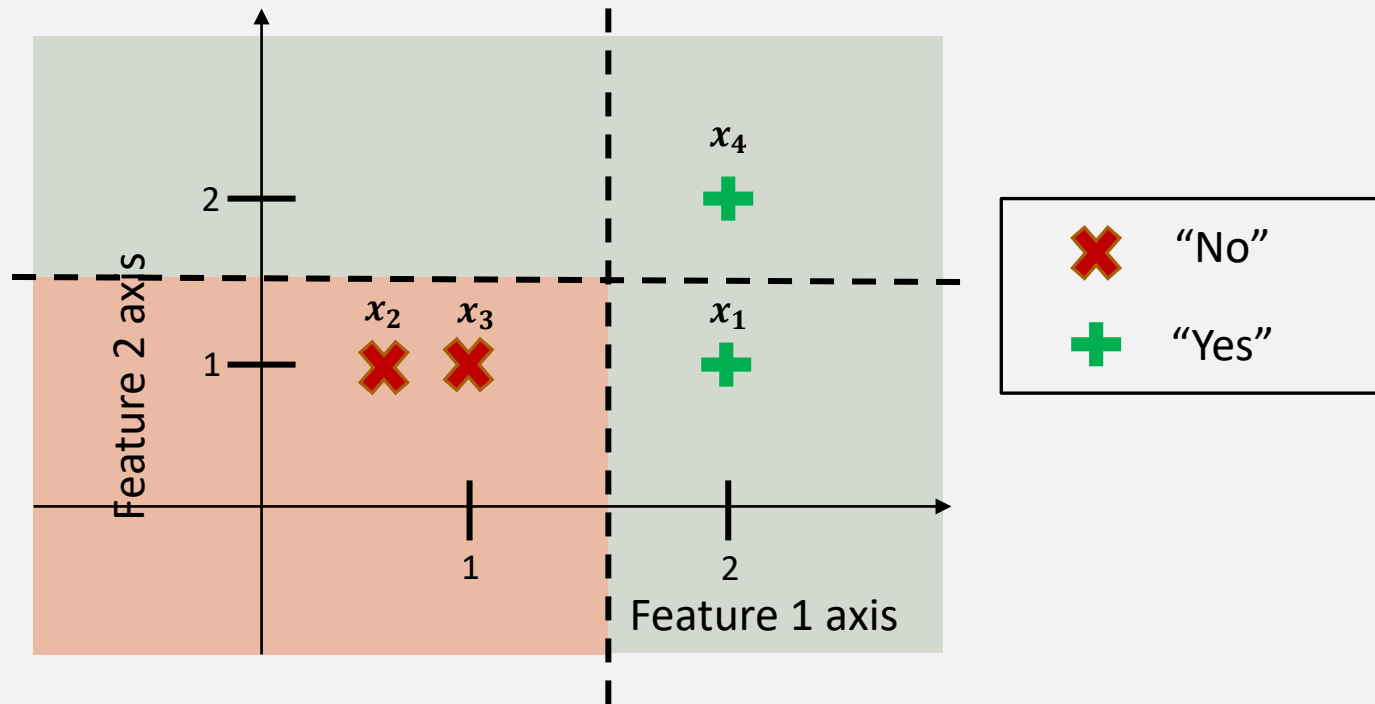
Decision Tree Example with cont. features

	Feature 1	Feature 2	Label
x_1	2	1	Yes
x_2	0.5	1	No
x_3	1	1	No
x_4	2	2	Yes



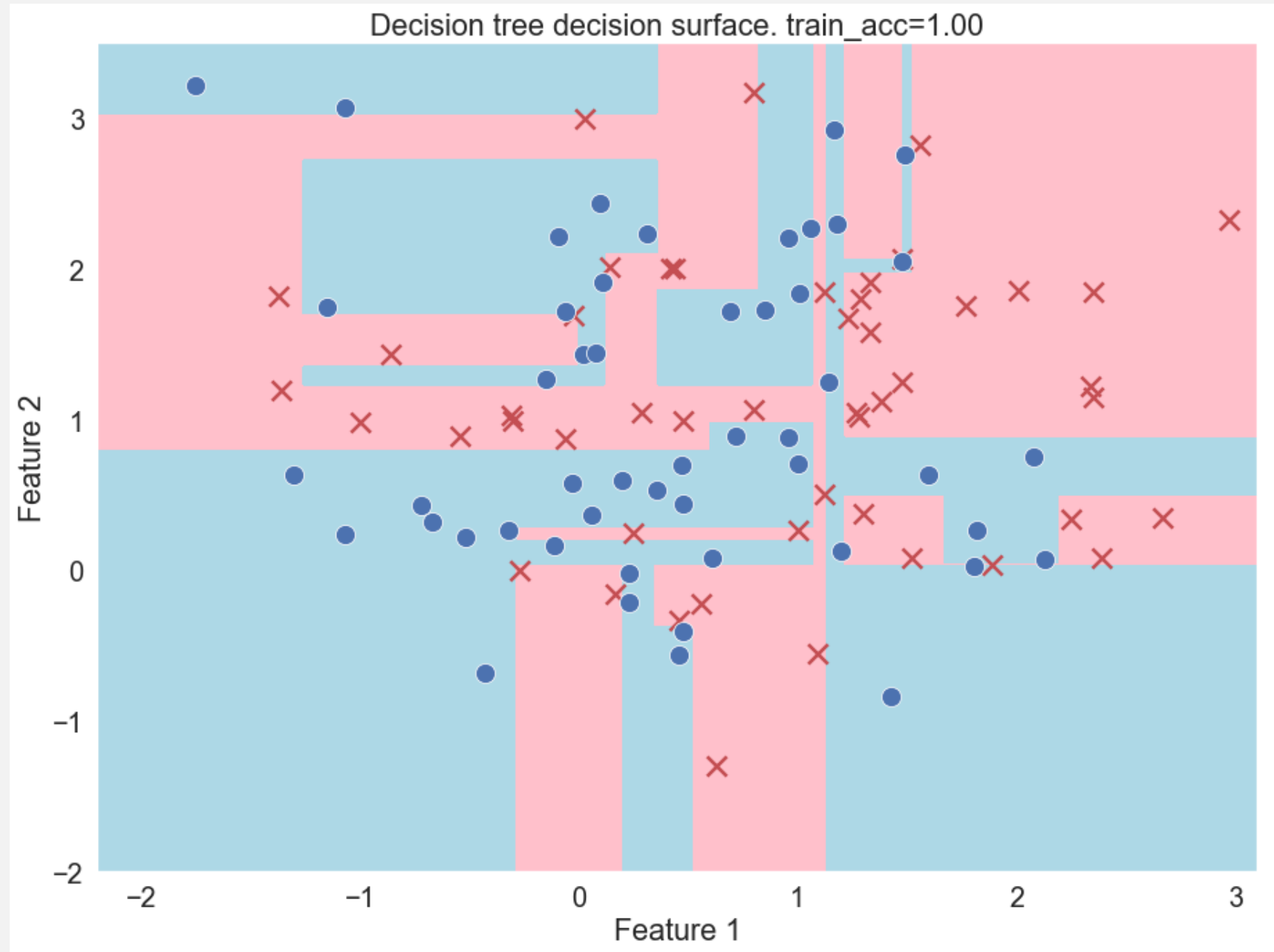
- Continuous features can be split on thresholds.
- Note you can find a smaller tree that fits this training data.

Decision Tree decision surface



- The decision surface of decision trees are axis-parallel rectangles based on the feature splits.
- In this plot, the two regions are defined by the split $\text{feature1} \leq 1.5$, then another two regions are defined by the split $\text{feature2} \leq 1.5$.

Decision Tree
decision surface



Decision Tree Learning

- DTs are primarily used for classification problems. Can be adapted for use with regression
- We will focus on classification decision trees. Works with binary and multiclass without modifications needed.
- Given a training set $\mathbf{T} = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$, build a decision tree that best fits this training data.
- Internal nodes are decisions that split on a feature.
- Leaf nodes contain the classification (prediction).
- Decision trees are built using greedy top-down approaches. Also known as greedy divide-and-conquer.

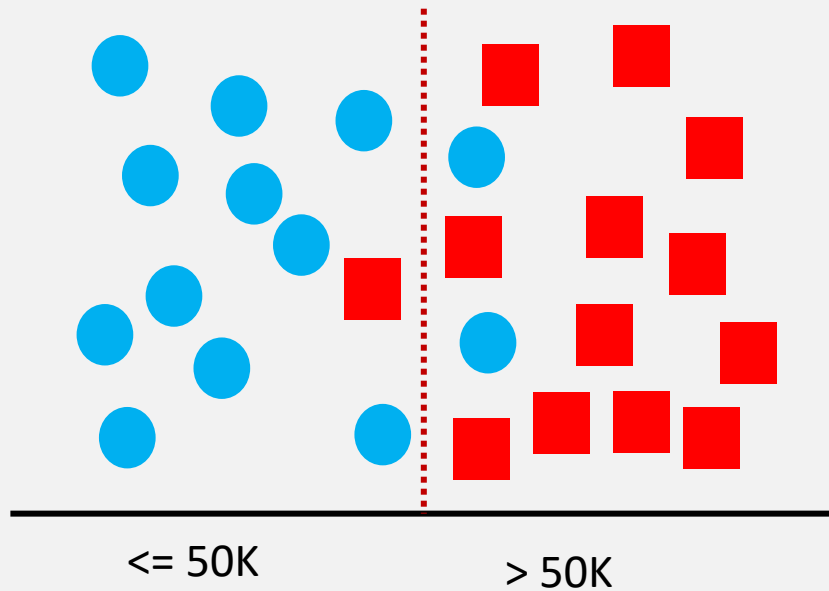
Building DT with discrete features only

Decision Tree Building – Selecting a good feature split [1]

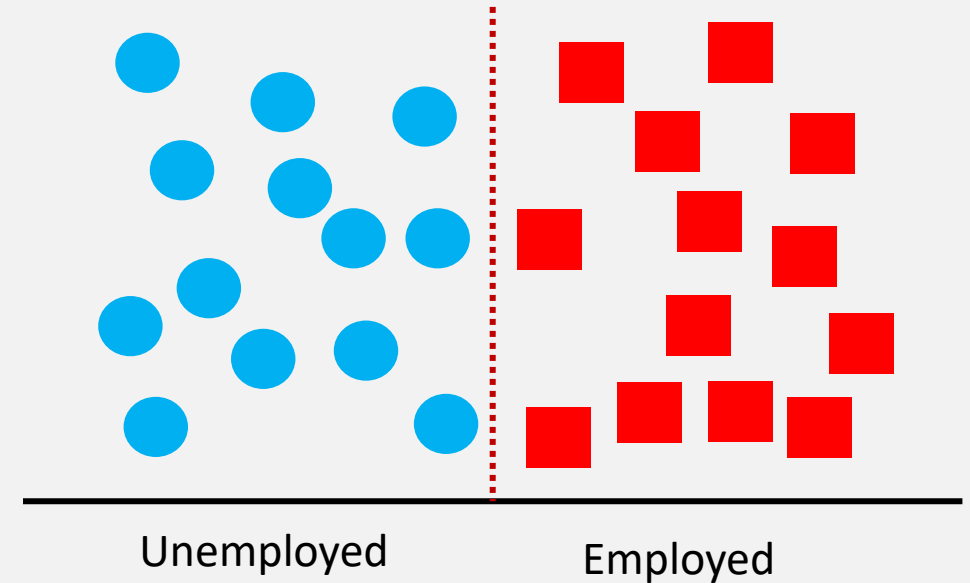
- Occam's Razor rule: the simplest hypothesis that fits the data is probably the correct one.
- Applying this rule to DT building, we want the smallest decision tree that fits the training data.
- But finding the minimal (smallest) decision tree consistent with the data is computationally demanding (NP-hard).
- An alternative approach is to use top-down divide-and-conquer methods that are not computationally demanding.
- These are greedy techniques that build the tree from top to bottom. At each step, it selects a feature to split on that results in best discrimination between the classes.
- This is a greedy heuristic search for a simple (small) tree. Not guaranteed to be the smallest tree, but we are okay with that. Doing it this way, it is less computationally demanding.

Decision Tree Building – Selecting a good feature split [2]

- We want to determine which feature split is most useful for discriminating between the classes.
- Which feature split is more informative for discriminating between the two classes?



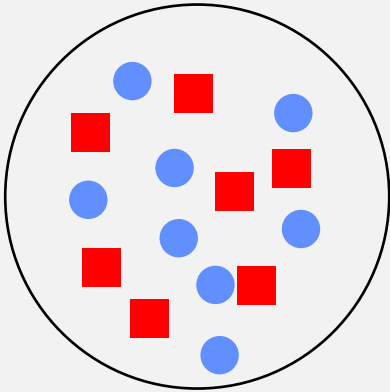
Split over whether Balance exceeds 50K



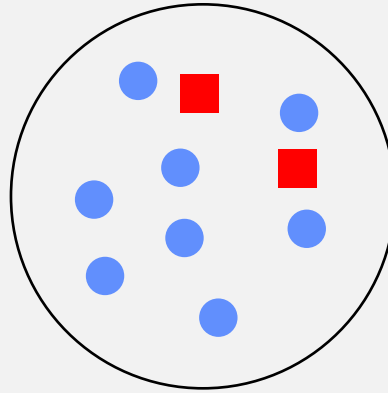
Split over whether applicant is employed

Decision Tree Building – Measuring Impurity of a Dataset [1]

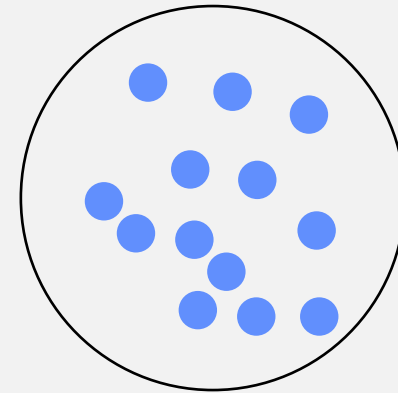
- Given a dataset for a classification task, there are many ways to measure impurity.



Very impure group



Less impure



Minimum impurity (pure)

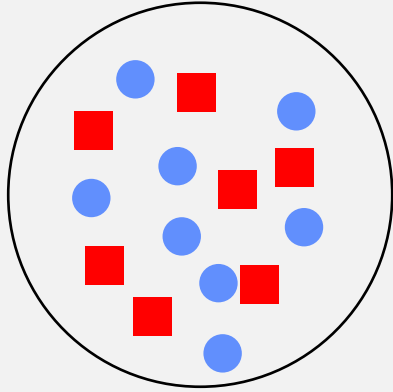
Decision Tree Building – Measuring Impurity of a Dataset [2]

- One popular impurity measure is called the **Gini impurity**.
- Suppose the classification task has $\{1, 2, 3, \dots, C\}$ classes, and we have a labelled dataset S with the fraction (probability) of each class given by p_i , then the Gini impurity of S is given by:

$$GI(S) = 1 - \sum_{i=1}^C p_i^2$$

- Note that p_i of each class is estimated from the dataset S .
- A **pure** dataset S containing samples of a single class would give $GI(S) = 0$.

Decision Tree Building – Measuring Impurity of a Dataset [3]

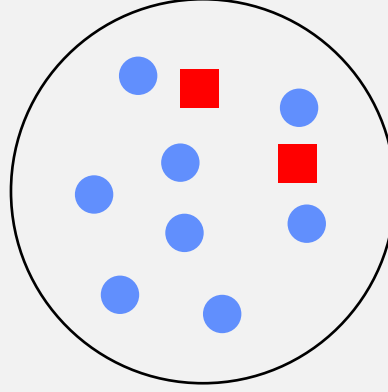


Very impure group

$$p_{red} = \frac{7}{15} = 0.47$$

$$p_{blue} = \frac{8}{15} = 0.53$$

$$\begin{aligned} GI(S) &= 1 - (p_{red}^2 + p_{blue}^2) \\ &= 1 - (0.47^2 + 0.53^2) \\ &= 0.4982 \end{aligned}$$

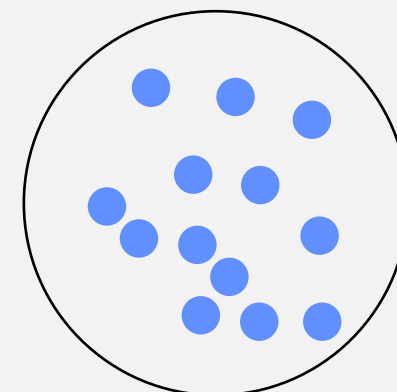


Less impure

$$p_{red} = \frac{2}{10} = 0.2$$

$$p_{blue} = \frac{8}{10} = 0.8$$

$$\begin{aligned} GI(S) &= 1 - (p_{red}^2 + p_{blue}^2) \\ &= 1 - (0.2^2 + 0.8^2) \\ &= 0.32 \end{aligned}$$



Minimum impurity (pure)

$$p_{red} = \frac{0}{13} = 0$$

$$p_{blue} = \frac{13}{13} = 1$$

$$\begin{aligned} GI(S) &= 1 - (p_{red}^2 + p_{blue}^2) \\ &= 1 - (0^2 + 1^2) \\ &= 0 \end{aligned}$$

- Notice that for a binary class, Gini impurity ranges from 0 (pure) to 0.5 (most impure).

Decision Tree Building – Measuring Impurity of a Dataset [4]

- If a data set S is split on an attribute A into K subsets $S_1, S_2, \dots, S_K$, the Gini Impurity of splitting on A can be defined as:

$$GI_A(S) = \frac{|S_1|}{|S|} GI(S_1) + \frac{|S_2|}{|S|} GI(S_2) + \dots + \frac{|S_K|}{|S|} GI(S_K)$$

- Note that if Q is a dataset, then $|Q|$ is the size of the dataset Q .
- So, if we split on a feature A , we can compute the resulting Gini impurity as the weighted average of the resulting splits.
- Greedy approach: we want a feature split that gives the **minimum Gini impurity**.

Decision Tree Building – Back to the example

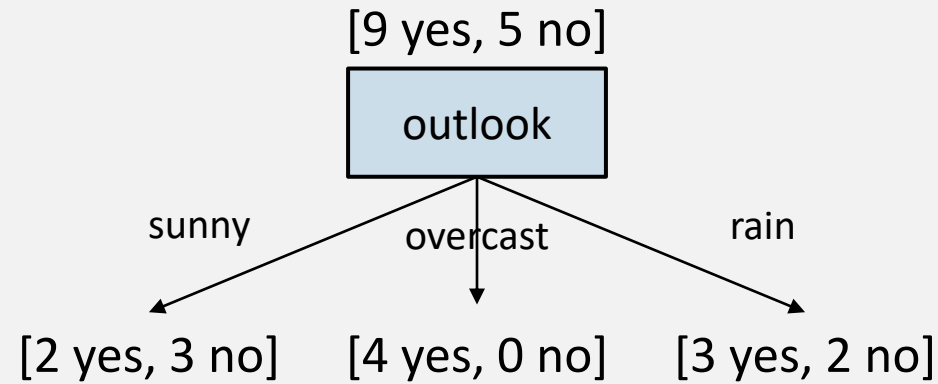
- Consider this training set T where the decision to play depends on the given features outlook, temp, humidity, and wind.

outlook	temp	humidity	wind	play
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

- How to build a decision tree that fits this data?
- Starting from root until you reach stop-condition:
 - Step 1: consider **remaining** features (outlook, temp, humidity, wind) for splitting.
 - Step 2: compute Gini impurity of each feature split.
 - Step 3: Select feature split that results in **minimum Gini impurity**.
- At the start, we want to select the feature split at the **root** of the tree. So, let's do these three steps.

Decision Tree Building – Divide-and-Conquer [1]

outlook	temp	humidity	wind	play
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No



$$p_{yes} = \frac{2}{5} = 0.4$$

$$p_{no} = \frac{3}{5} = 0.6$$

$$\begin{aligned}
 GI_{sunny}(S_1) &= 1 - (p_{yes}^2 + p_{no}^2) \\
 &= 1 - (0.4^2 + 0.6^2) \\
 &= 0.48
 \end{aligned}$$

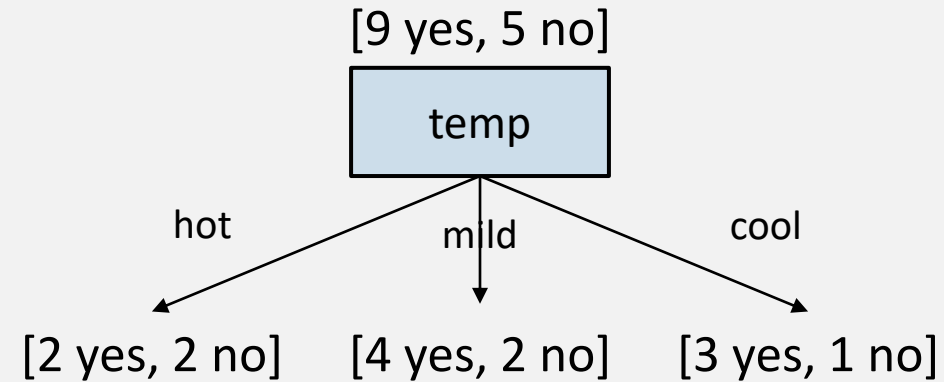
$$\begin{aligned}
 GI_{overcast}(S_2) &= 1 - (1^2 + 0^2) \\
 &= 0
 \end{aligned}$$

$$\begin{aligned}
 GI_{rain}(S_3) &= 1 - (0.6^2 + 0.4^2) \\
 &= 0.48
 \end{aligned}$$

$$\begin{aligned}
 GI_{outlook}(S) &= \frac{5}{14} GI_{sunny} + \frac{4}{14} GI_{overcast} + \frac{5}{14} GI_{rain} \\
 &= \frac{5}{14} 0.48 + \frac{4}{14} 0 + \frac{5}{14} 0.48 = 0.34
 \end{aligned}$$

Note that you take the weighted average of the splits.

Decision Tree Building – Divide-and-Conquer [2]



outlook	temp	humidity	wind	play
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

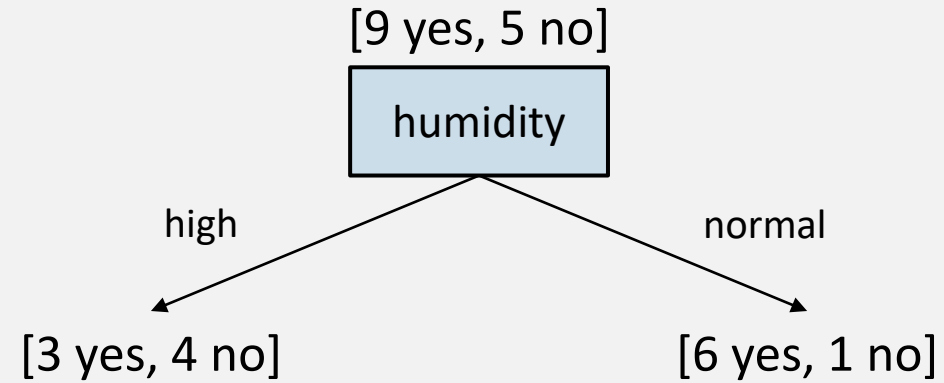
$$GI_{hot}(S_1) = 1 - (0.5^2 + 0.5^2) = 0.5$$

$$GI_{mild}(S_2) = 1 - (0.67^2 + 0.33^2) = 0.4422$$

$$GI_{cool}(S_3) = 1 - (0.75^2 + 0.25^2) = 0.375$$

$$GI_{temp}(S) = \frac{4}{14} 0.5 + \frac{6}{14} 0.4422 + \frac{4}{14} 0.375 = 0.440$$

Decision Tree Building – Divide-and-Conquer [3]



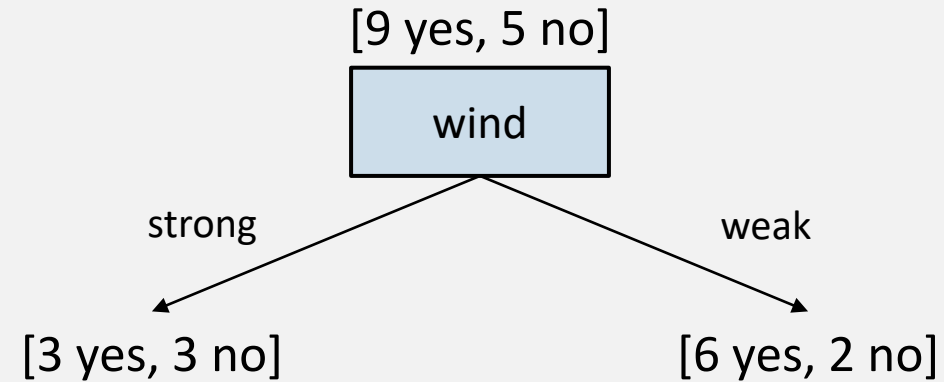
outlook	temp	humidity	wind	play
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

$$GI_{high}(S_1) = 1 - (0.43^2 + 0.57^2) = 0.4902$$

$$GI_{normal}(S_2) = 1 - (0.86^2 + 0.14^2) = 0.2408$$

$$GI_{humidity}(S) = \frac{7}{14} 0.4902 + \frac{7}{14} 0.2408 = 0.3655$$

Decision Tree Building – Divide-and-Conquer [4]



outlook	temp	humidity	wind	play
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

$$GI_{strong}(S_1) = 1 - (0.5^2 + 0.5^2) = 0.5$$

$$GI_{weak}(S_2) = 1 - (0.75^2 + 0.25^2) = 0.375$$

$$GI_{wind}(S) = \frac{6}{14} 0.5 + \frac{8}{14} 0.375 = 0.4286$$

Decision Tree Building – Divide-and-Conquer [5]

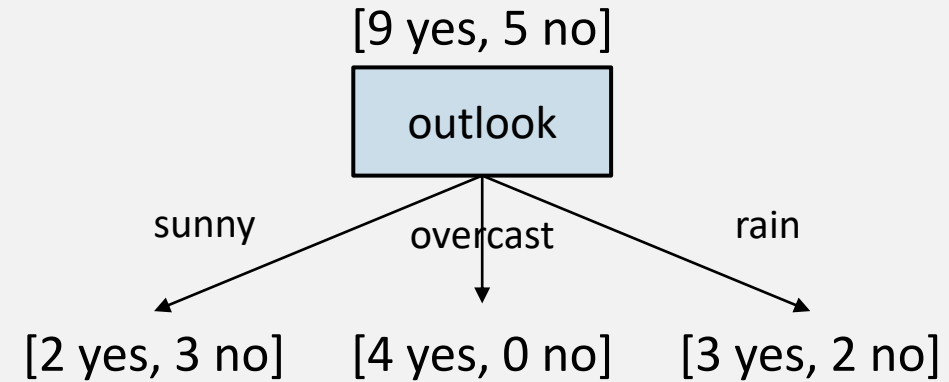
- Pick feature split that has min gini impurity

$$GI_{outlook}(S) = 0.34$$

$$GI_{temp}(S) = 0.440$$

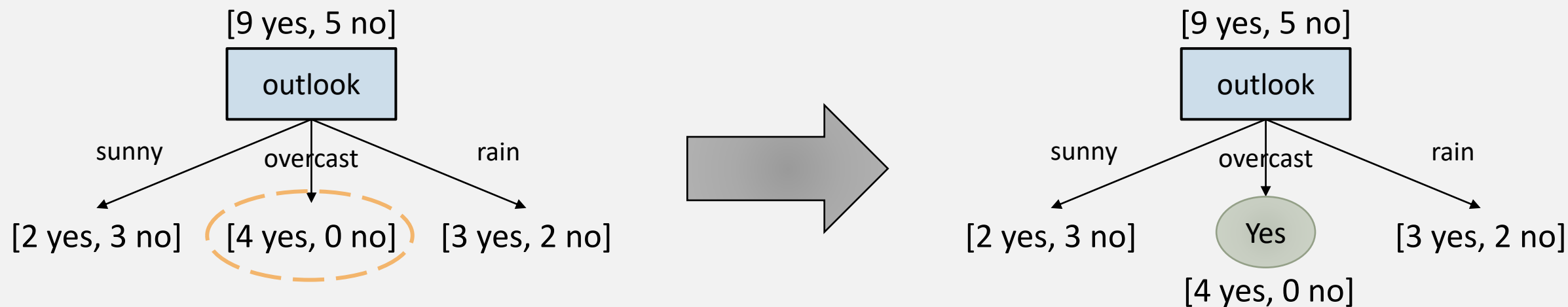
$$GI_{humidity}(S) = 0.3655$$

$$GI_{wind}(S) = 0.4286$$



- outlook has the minimum Gini impurity after the split. So, we split on this feature and continue building the tree.
- We recursively apply tree building on each subtree (split) until we reach a stop condition for a node.
- A stop condition can be until a fixed depth is reached, all instances in the node belong to same class, or only a few instances are in the node.

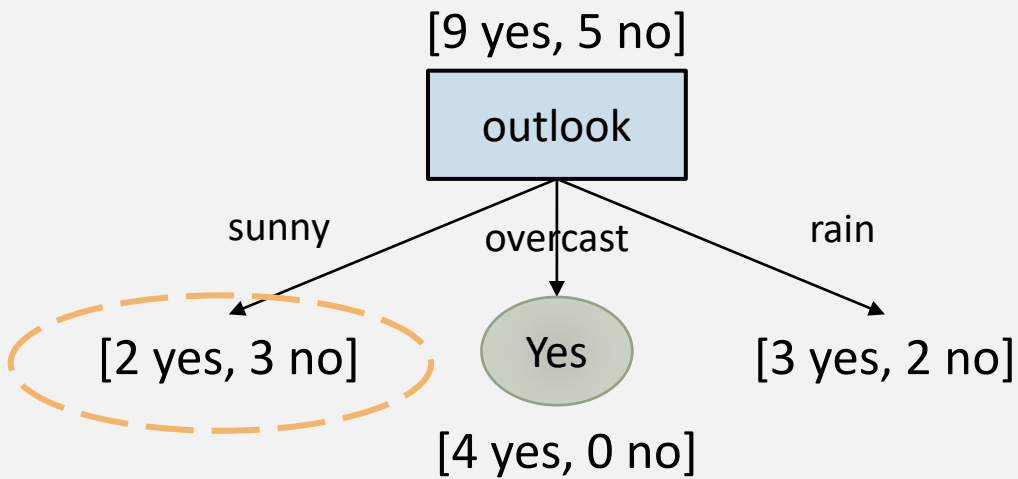
Decision Tree Building – Divide-and-Conquer [6]



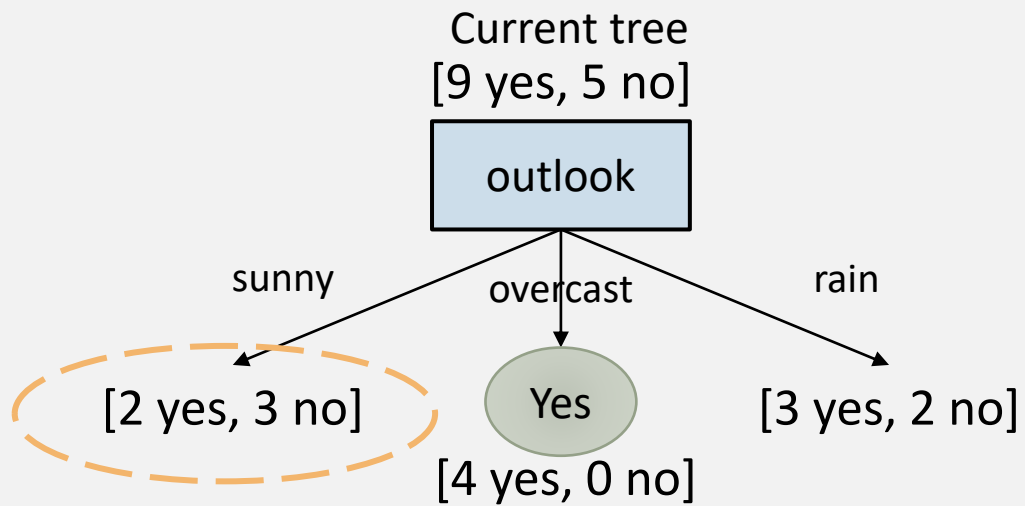
- When a node has instances only from one class. We make it a leaf node and it outputs the class value.
- No need to continue splitting the dataset at that leaf node.

Decision Tree Building – Divide-and-Conquer [7]

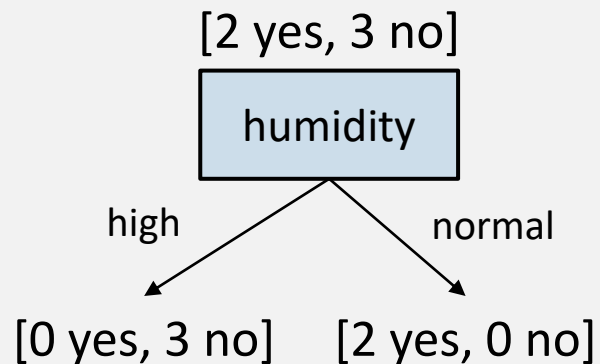
- Continue tree building for remaining nodes with impure data.
- Step 1: consider remaining features (temp, humidity, wind) for splitting.
- Step 2: compute the Gini impurity of each feature split.
- Step 3: Select feature that results in minimum Gini impurity.



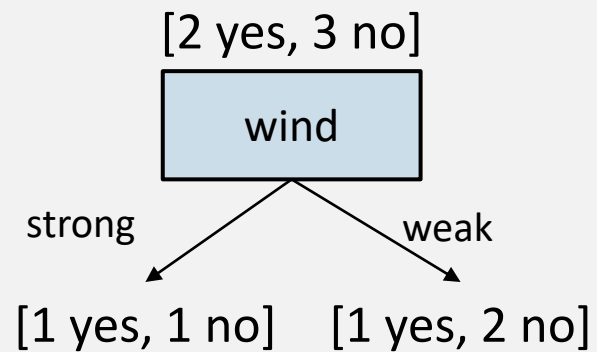
outlook	temp	humidity	wind	play
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No



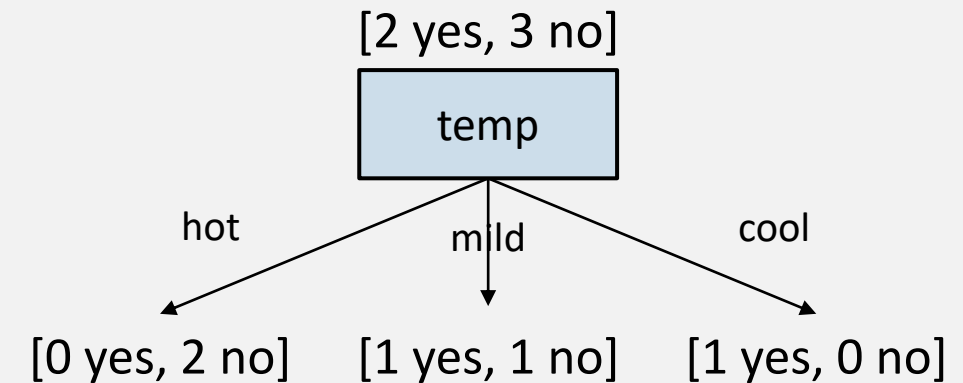
outlook	temp	humidity	wind	play
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes



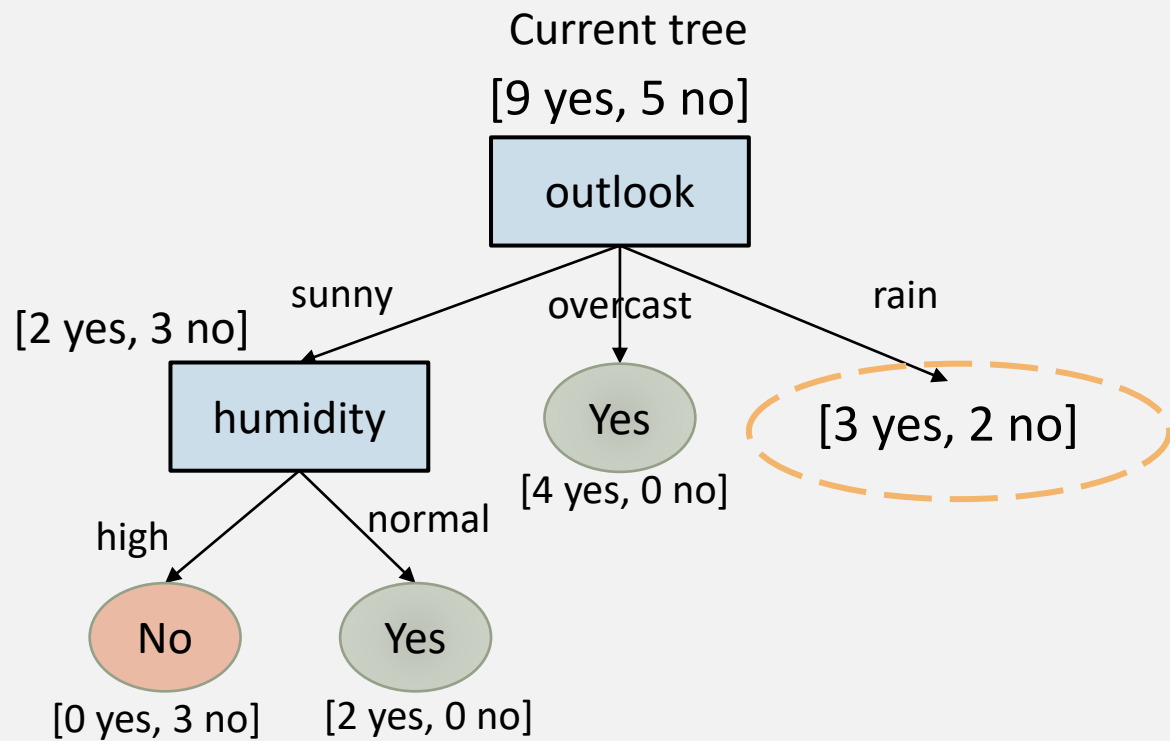
$$GI_{humidity}(S) = 0$$



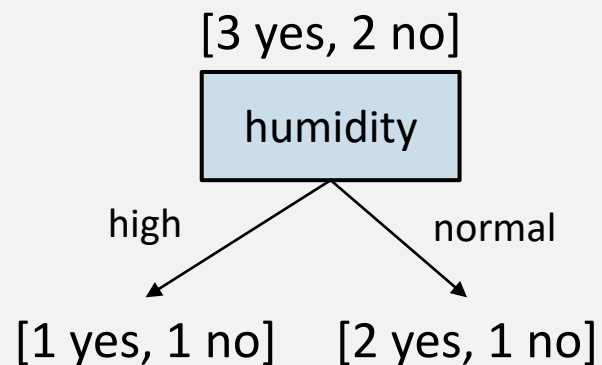
$$GI_{wind}(S) = 0.2 + 0.2653 = 0.4653$$



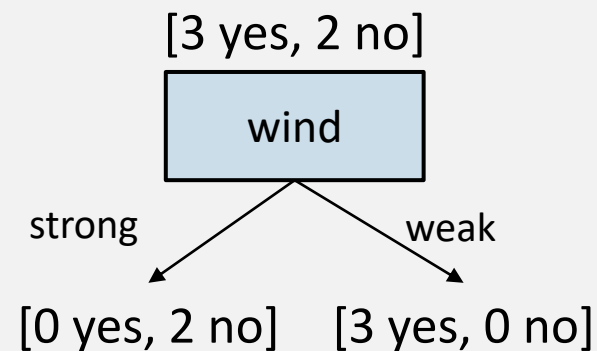
$$GI_{temp}(S) = 0.2$$



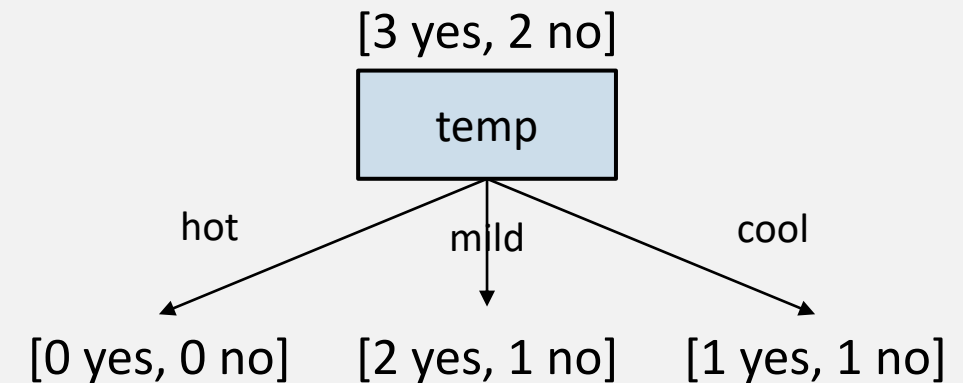
outlook	temp	humidity	wind	play
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Rain	Mild	Normal	Weak	Yes
Rain	Mild	High	Strong	No



$$GI_{humidity}(S) = 0.4653$$



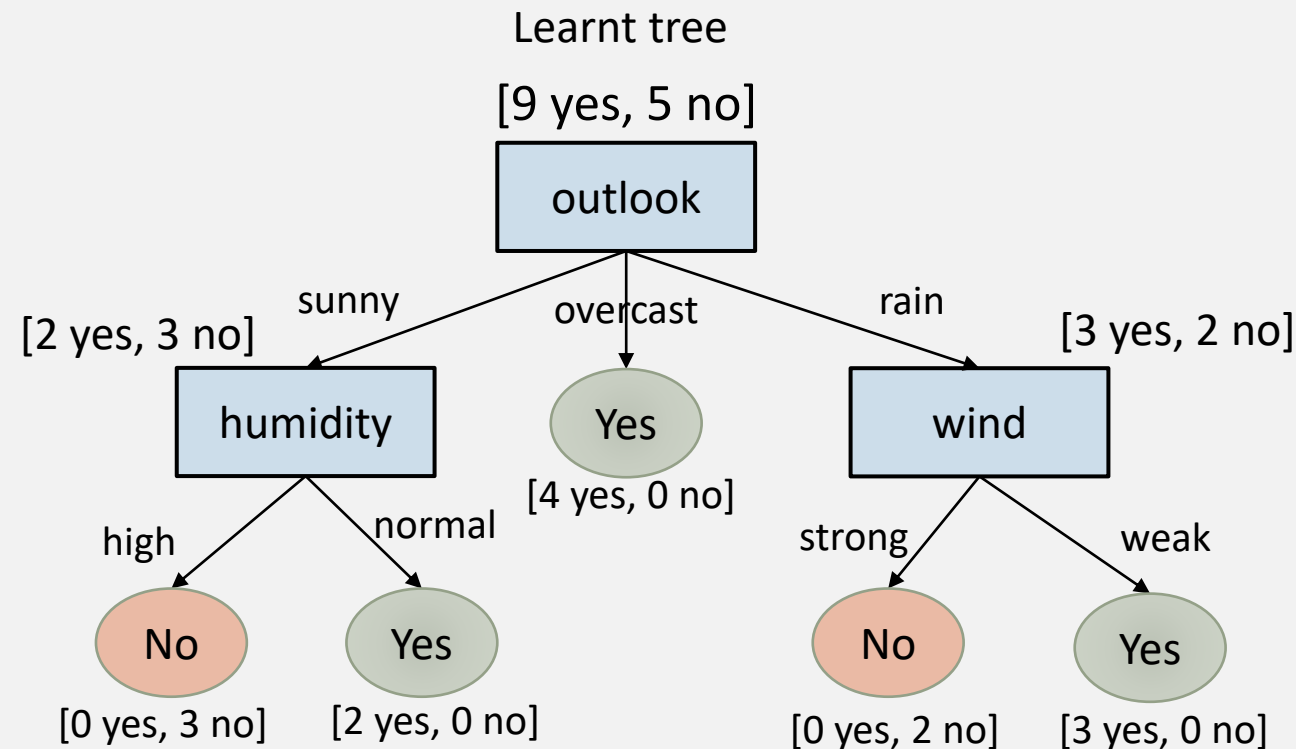
$$GI_{wind}(S) = 0$$



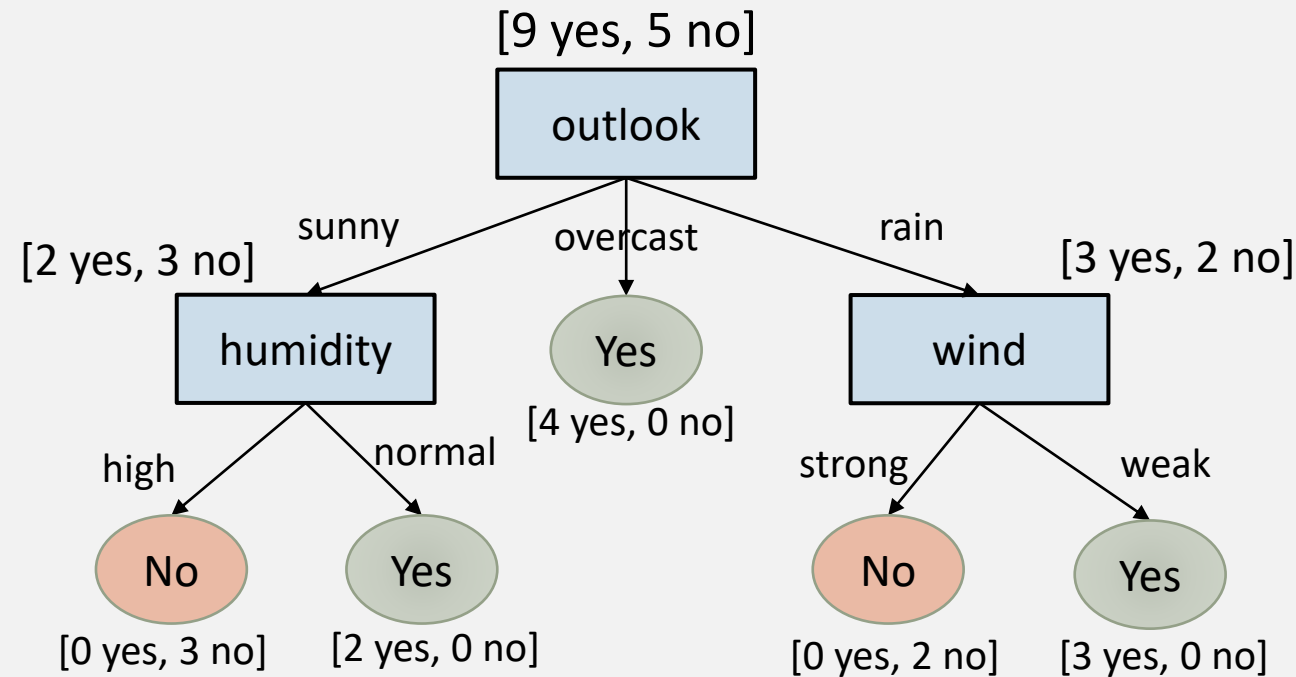
$$GI_{temp}(S) = 0.4653$$

Decision Tree Building – Divide-and-Conquer [8]

- There are no more non-leaf nodes. Now we are done building the tree.
- What is the tree prediction on $x_{new} = [sunny, hot, normal, strong]$
- $y_{new} = Yes$



Decision Tree Building – Divide-and-Conquer [10]



- We stop further tree building since all nodes have instances from same class.
- DT algorithms can maintain statistics of each node's instances; how many instances from each class fall into a node. This allows for computing probabilities.
- Some DT algorithms will create a leaf node when a specified number of instances are in a node (for example 4 instances). So, leaf nodes can be impure. In this case, the classification can be the majority vote or probability output.

MakeSubtree(set of training instances D)

A subtree routine that splits the data D at the CURRENT node, we call this routine for every non-leaf node.

$C = \text{DetermineCandidateSplits}(D)$

Which features should be considered for splitting? This includes the continuous feature threshold splits.

if stopping criteria met

make a leaf node N

determine class label/probabilities for N

Check if a stopping condition is met like all instances in D are from a single class.

else

make an internal node N

$S = \text{FindBestSplit}(D, C)$

Given the candidate feature splits C and data D , compute the Gini Impurity for each split candidate, and return the split S that results in min Gini Impurity

for each outcome k of S

D_k = subset of instances that have outcome k

Split the data into nodes based on feature value

k^{th} child of $N = \text{MakeSubtree}(D_k)$

Call the tree building subroutine on each child node.

return subtree rooted at N

Figure from: [link](#)

Strengths of Decision Trees

- Many variations of DT algorithms. Which features to consider for splitting, the impurity measure used, stopping criteria, max depth, etc.
- Popular DT algorithms: ID3 (Iterative Dichotomiser 3), C4.5 (successor of ID3), CART (Classification And Regression Tree). The tree building process we've shown in the slides is similar to CART.
- scikit-learn [`sklearn.tree.DecisionTreeClassifier`](#) is *similar* to CART.
- Learns an explicit hypothesis in the form of decisions. This is great if interpretation of what the model has learned is important.
- Works with discrete string features. Works with continuous features.
- Does not need feature scaling (unaffected by it).
- Implementations like scikit-learn are fast.
- Allows for ranking of features based on their importance; the features at the top of the built tree are the most informative.

Limitations of Decision Trees

- Sensitive to noise in the data. A small change in the training data can result in a very different tree.
- Tends to overfit the training data. There are various methods for combatting overfitting in decision trees.
- Other ML algorithms usually outperform DT algorithms (due to tendency of overfitting). However, when used as a base model in a random forest or ensemble, results in a powerful generalizable model.

```
class sklearn.tree.DecisionTreeClassifier(*, criterion='gini',
splitter='best', max_depth=None, min_samples_split=2, min_samples_leaf=1,
min_weight_fraction_leaf=0.0, max_features=None, random_state=None,
max_leaf_nodes=None, min_impurity_decrease=0.0, class_weight=None,
ccp_alpha=0.0, monotonic_cst=None) \[source\]
```

max_depth : int, default=None

The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than min_samples_split samples.

min_samples_split : int or float, default=2

The minimum number of samples required to split an internal node:

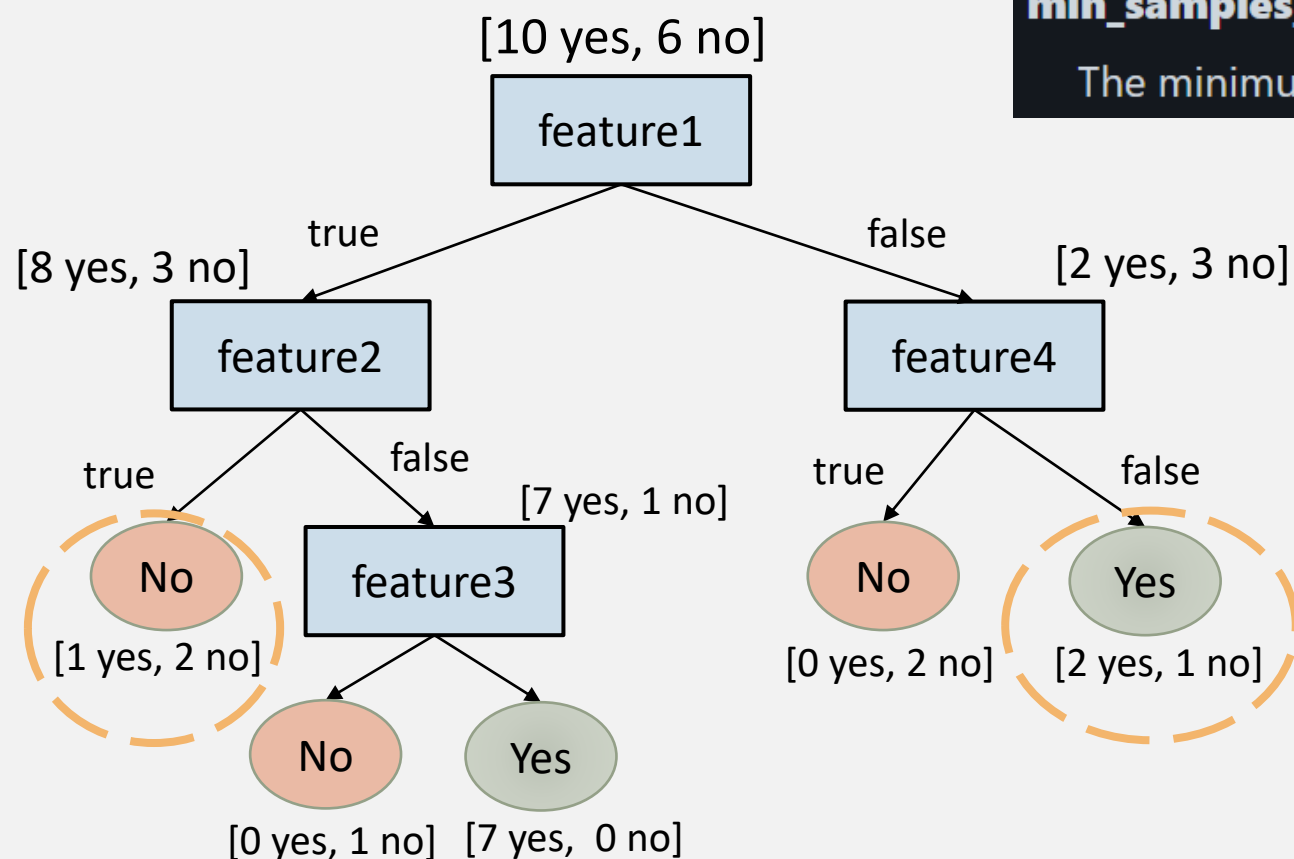
min_samples_leaf : int or float, default=1

The minimum number of samples required to be at a leaf node. A split point at any depth will only be considered if it leaves at least `min_samples_leaf` training samples in each of the left and right branches. This may have the effect of

- See [documentation](#) at scikit-learn website.
- Some important parameters that change stop-condition: max_depth, min_samples_split, and min_samples_leaf.
- Nice discussion on difference between min_samples_split and min_samples_leaf here [link](#).

Sklearn DT stop-conditions

- Some important parameters that change stop-condition: `max_depth`, `min_samples_split`, and `min_samples_leaf`.
- This may help combat overfitting as it results in smaller trees, and *smoother* decisions.



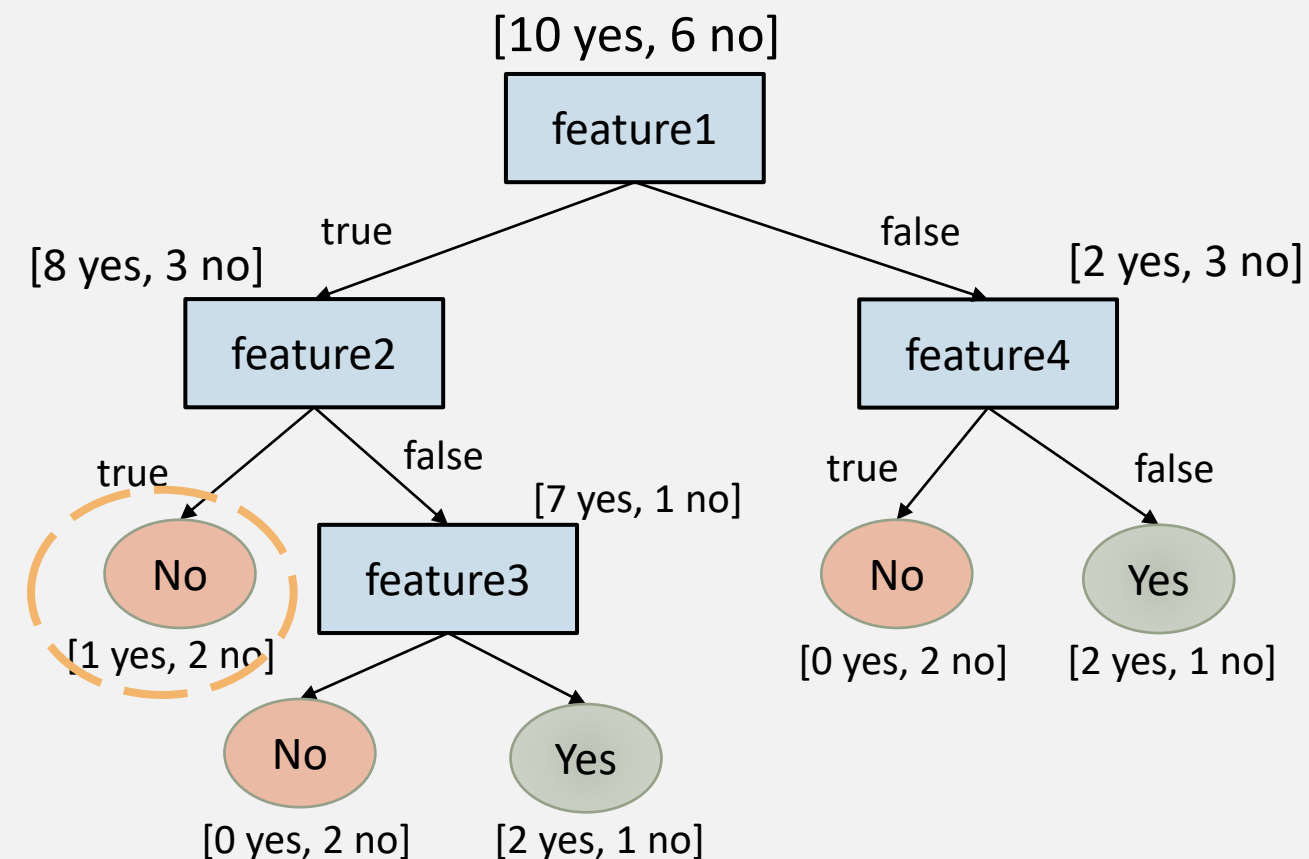
`min_samples_split` : *int or float, default=2*

The minimum number of samples required to split an internal node:

With `min_samples_split=4`, the two highlighted nodes were made leaves and given the majority vote label. Even though they are impure.

Predicting probabilities of a label

- We can maintain statistics on the training instances that end up at each node.
- This information can be used to output a class probability prediction.



- Given $x_{new} = [true, true, false, false]$
- This instance will end up in the [1 yes, 2 no] leaf node.
- We can predict a class based on majority vote: $y_{new} = no$
- We can also predict the probability of yes: $y_{new} = Prob(yes) = \frac{1}{3} = 0.3333$

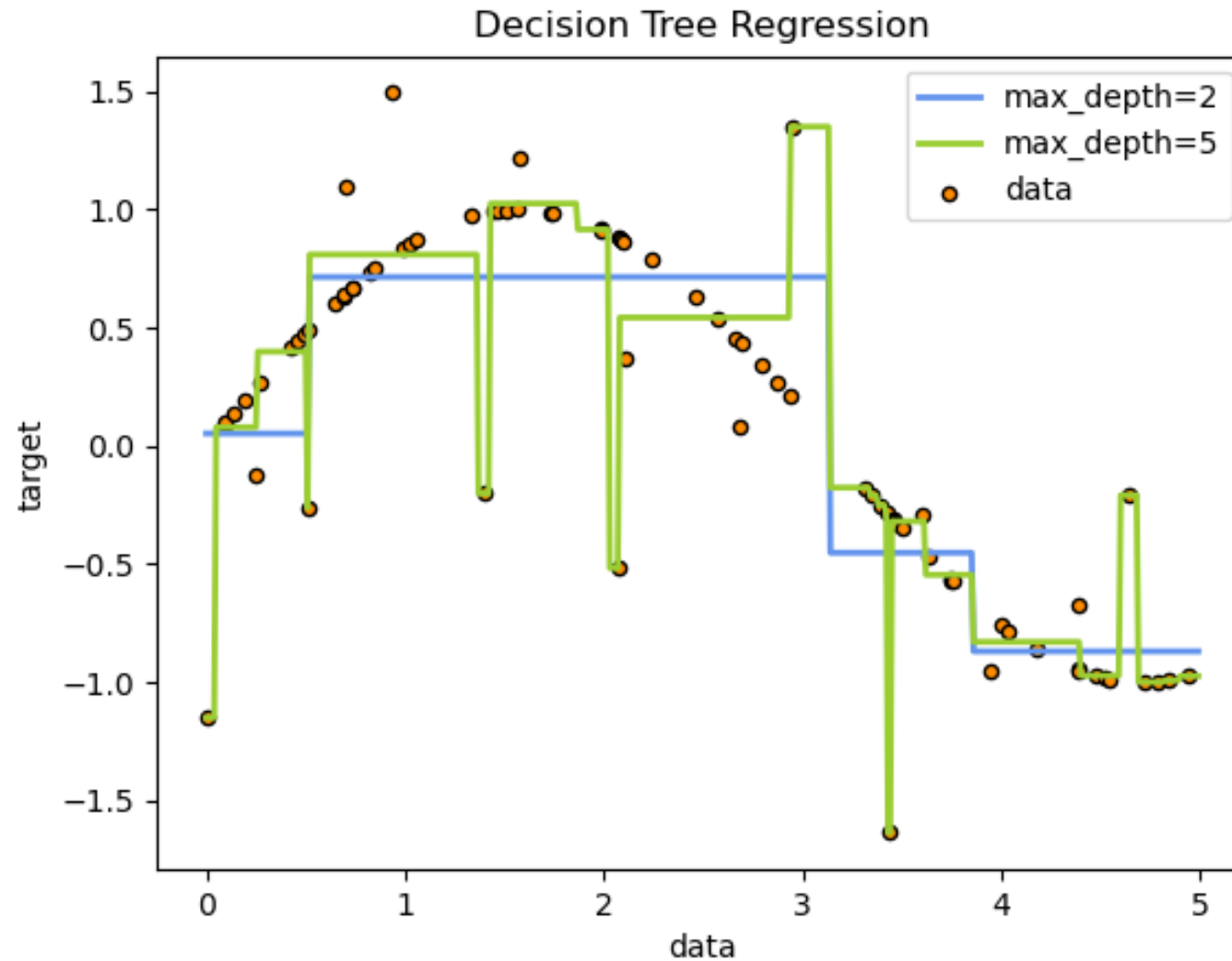
Regression Trees

- We won't go into detail on how to build regression trees.
- However, it is similar to how we build classification trees with minor changes.
- In regression trees, a node is assigned the average label value of the training instances that fall into that node.
- Instead of impurity measure for choosing which feature to split on, we use a loss function. A feature that gives a lower loss value is preferred.
- A common loss function used is the squared difference:

$$Loss_A(S) = \sum_{D \in splits(S)} \sum_{i \in D} (y_i - \mu_D)^2$$

The feature A splits the data S into child-nodes. In each child-node, we compute the squared difference between the instance labels y_i and the average label value μ_D . We then sum this loss over all children nodes. Compute this loss for every feature. Select the feature that gives the minimum loss.

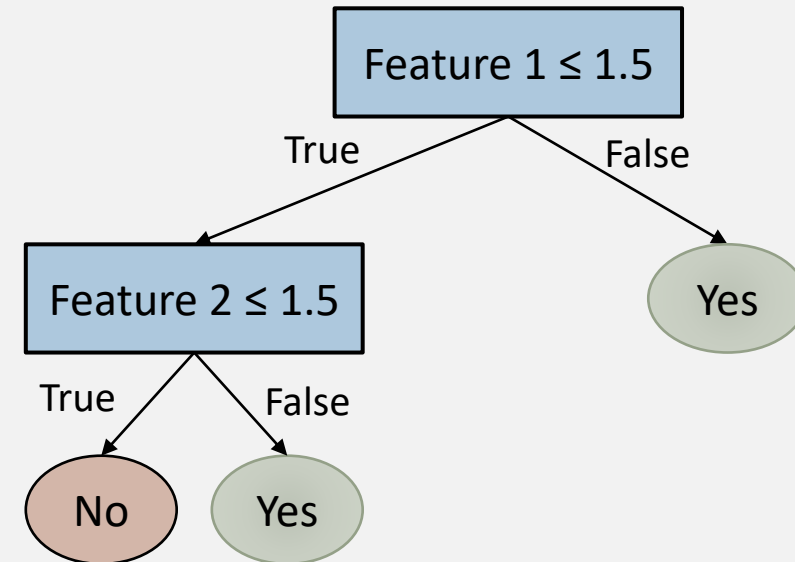
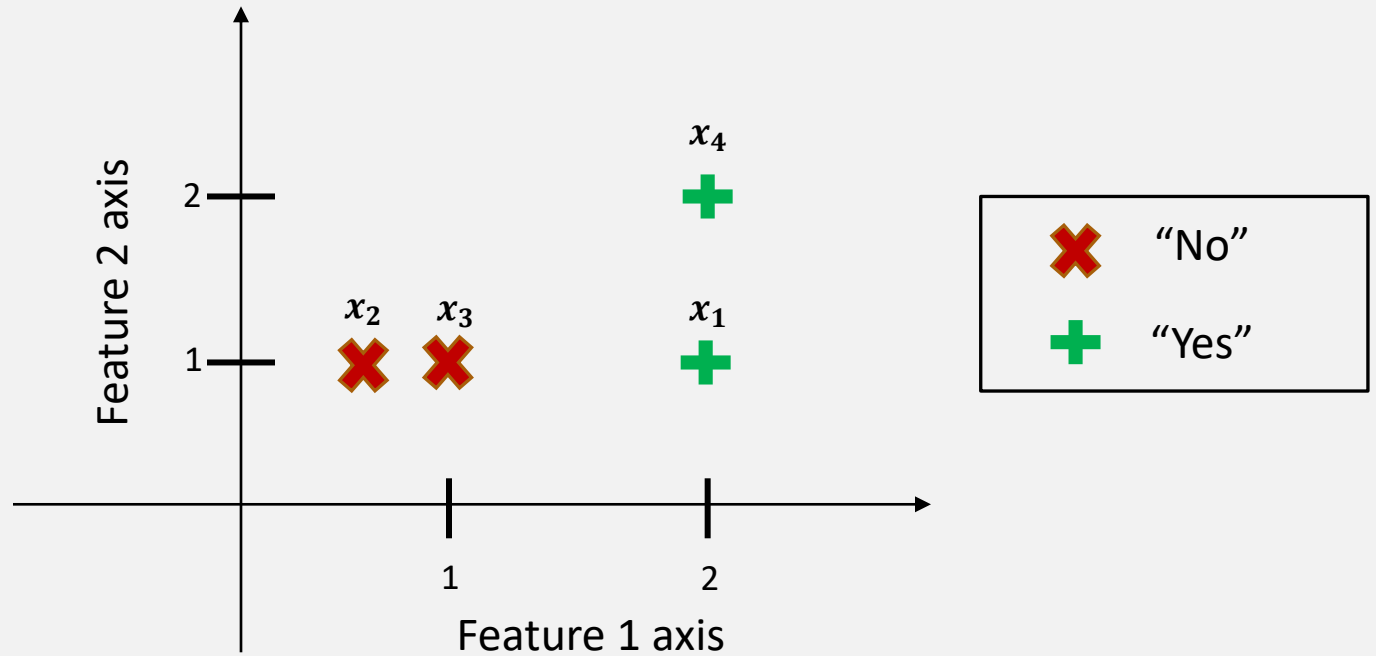
Regression Tree example with 1 feature dataset.



See [link](#)

Decision Tree with cont. features

	Feature 1	Feature 2	Label
x_1	2	1	Yes
x_2	0.5	1	No
x_3	1	1	No
x_4	2	2	Yes



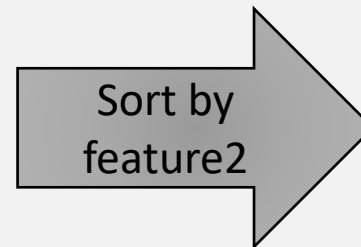
- Continuous features can be split on thresholds.
- We won't discuss

Building DT with continuous features

Decision Tree Building – Continuous features [1]

- Consider the dataset given below with discrete and continuous features.
- Same rules apply for tree building: consider feature1 and feature2 for splitting, and select feature with minimum Gini impurity.
- How do we split on continuous features?
- Sort the instances by the continuous feature and compute the Gini impurity at the midpoints.

feature1	feature2	output
circle	-0.36	red
circle	15	green
circle	22	blue
circle	28	blue
square	3.1	green
square	32.1	blue
circle	39.8	blue
square	9	green
circle	-1.2	red
square	0.4	red

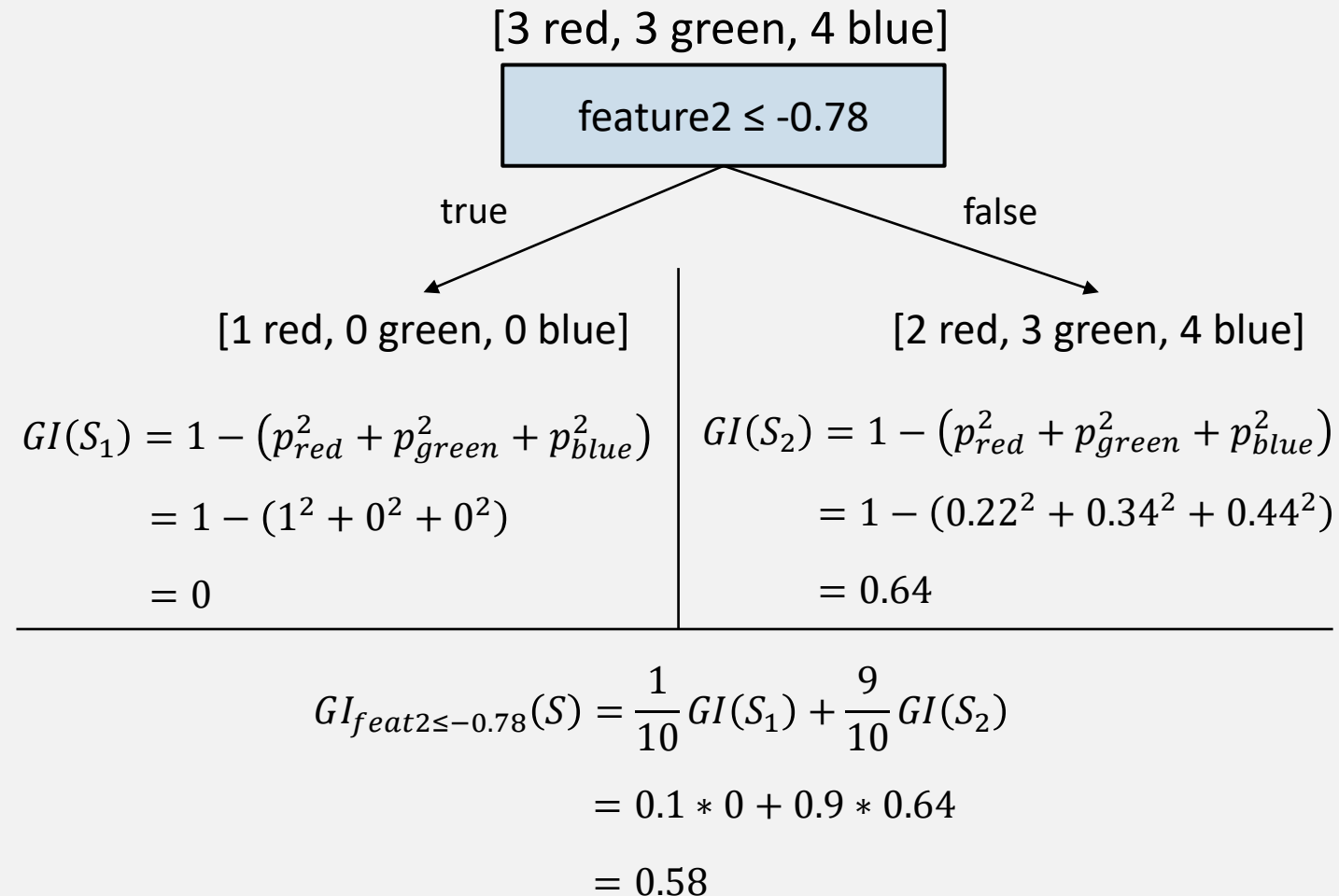


feature1	feature2	output
circle	-1.2	red
circle	-0.36	red
square	0.4	red
square	3.1	green
square	9	green
circle	15	green
circle	22	blue
circle	28	blue
square	32.1	blue
circle	39.8	blue

Decision Tree Building – Continuous features [2]

- Sort the instances by the continuous feature and compute the Gini impurity at the midpoints.

feature1	feature2	output	feature2 midpoints
circle	-1.2	red	-0.78
circle	-0.36	red	0.02
square	0.4	red	1.75
square	3.1	green	6.05
square	9	green	12
circle	15	green	18.5
circle	22	blue	25
circle	28	blue	30.05
square	32.1	blue	35.95
circle	39.8	blue	



Decision Tree Building – Continuous features [3]

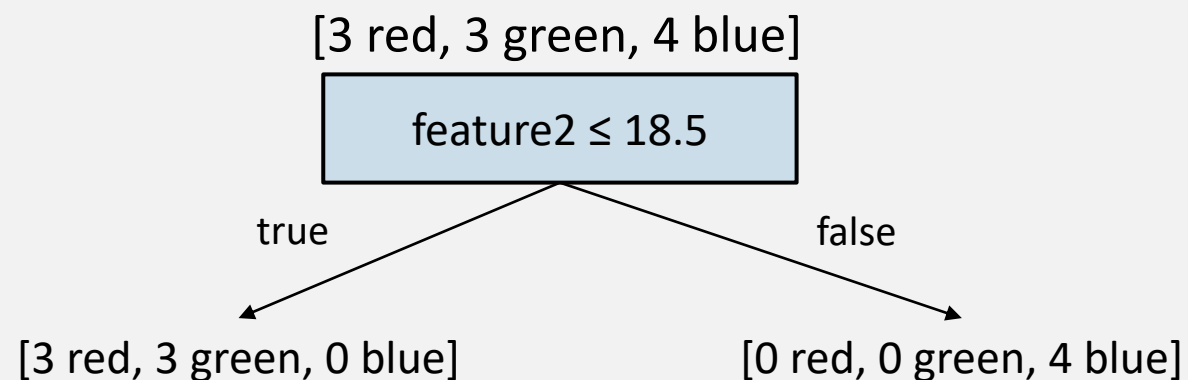
- Compute the Gini impurity at the midpoints. Find the split threshold with min Gini impurity.

feature1	feature2	output	feature2 midpoints		
circle	-1.2	red	-0.78	$GI_{feat2 \leq -0.78}(S) = 0.58$	
circle	-0.36	red	0.02	$GI_{feat2 \leq 0.02}(S) = 0.48$	
square	0.4	red	1.75	$GI_{feat2 \leq 1.75}(S) = 0.34$	
square	3.1	green	6.05	$GI_{feat2 \leq 6.05}(S) = 0.42$	
square	9	green	12	$GI_{feat2 \leq 12}(S) = 0.40$	
circle	15	green	18.5	$GI_{feat2 \leq 18.5}(S) = 0.30$	→ Split with min Gini impurity
circle	22	blue	25	$GI_{feat2 \leq 25}(S) = 0.43$	
circle	28	blue	30.05	$GI_{feat2 \leq 30.05}(S) = 0.52$	
square	32.1	blue	35.95	$GI_{feat2 \leq 35.95}(S) = 0.60$	
circle	39.8	blue			

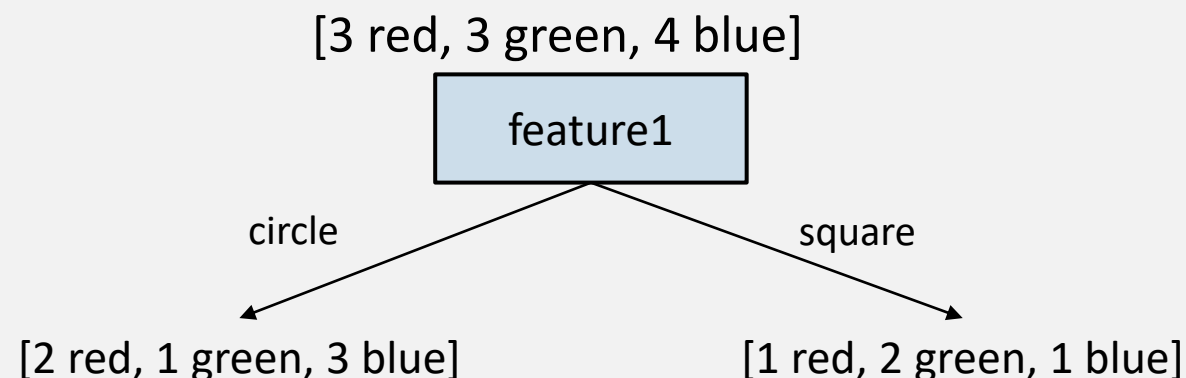
Decision Tree Building – Continuous features [4]

- So, we found the best split threshold for feature2. But we still need to compare it against other features; feature1 in this case.

feature1	feature2	output
circle	-1.2	red
circle	-0.36	red
square	0.4	red
square	3.1	green
square	9	green
circle	15	green
circle	22	blue
circle	28	blue
square	32.1	blue
circle	39.8	blue



$$GI_{feat2 \leq 18.5}(S) = 0.30$$

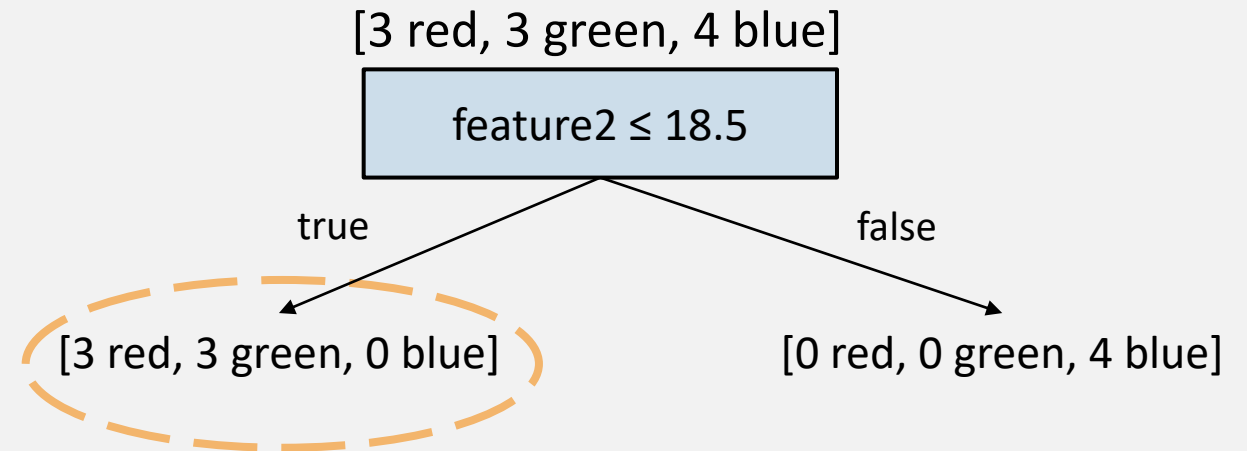


$$GI_{feature1}(S) = 0.62$$

Decision Tree Building – Continuous features [5]

- So, we split on $\text{feature2} \leq 18.5$ since it gives min Gini impurity.
- Same as before, we proceed building the tree recursively for each child node.

feature1	feature2	output
circle	-1.2	red
circle	-0.36	red
square	0.4	red
square	3.1	green
square	9	green
circle	15	green
circle	22	blue
circle	28	blue
square	32.1	blue
circle	39.8	blue



Decision Tree Building – Continuous features [6]

- Consider remaining features (feature1 AND feature2).
- Note that we consider splitting on continuous features again.

feature1	feature2	output
circle	-1.2	red
circle	-0.36	red
square	0.4	red
square	3.1	green
square	9	green
circle	15	green

feature2
midpoints

-0.78 $G I_{feat2 \leq -0.78}(S) = 0.40$

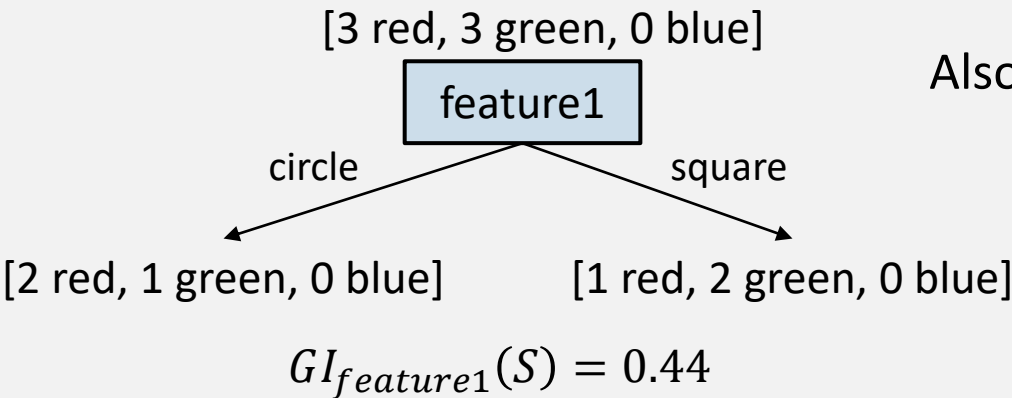
0.02 $G I_{feat2 \leq 0.02}(S) = 0.25$

1.75 $G I_{feat2 \leq 1.75}(S) = 0$

6.05 $G I_{feat2 \leq 6.05}(S) = 0.25$

12 $G I_{feat2 \leq 12}(S) = 0.40$

Split threshold with
min Gini impurity

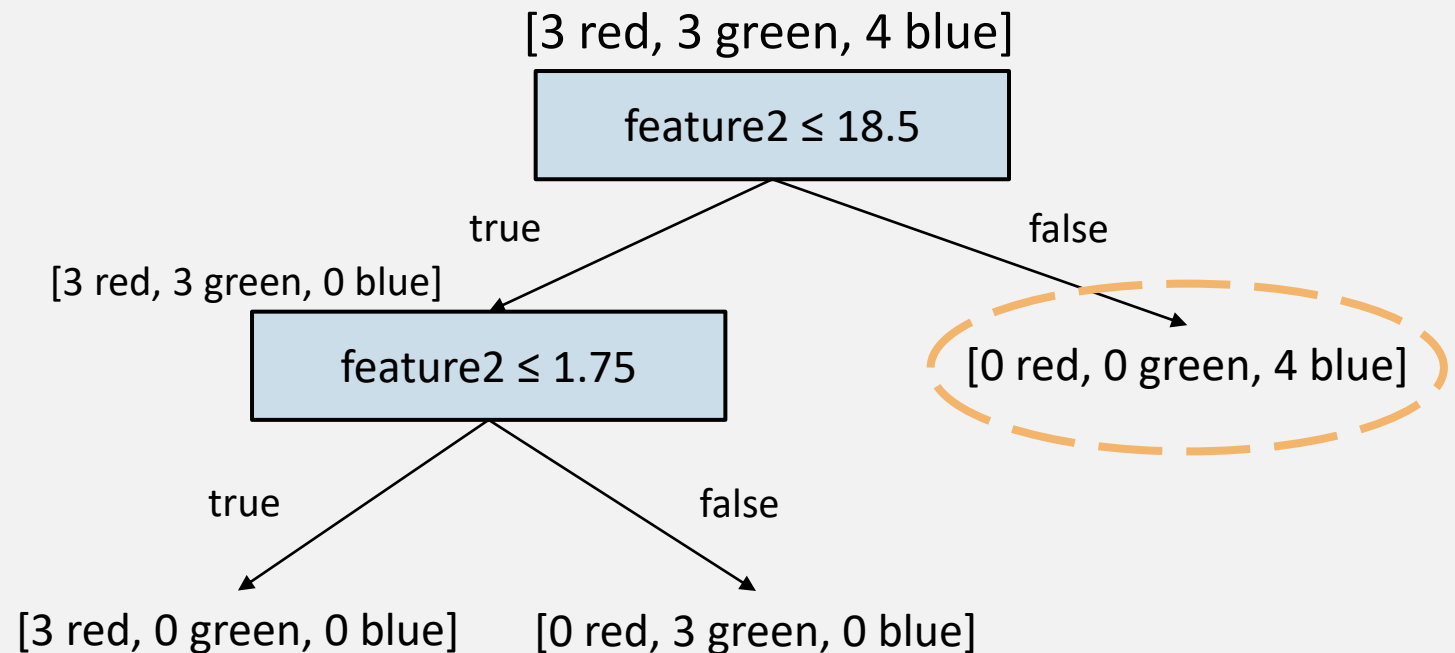


Also need to consider feature1

Decision Tree Building – Continuous features [7]

- So, we split on $\text{feature2} \leq 1.75$ since it gives min Gini impurity.
- Same as before, we proceed building the tree recursively for next non-leaf node.
- The highlighted node has instances from single class so we make it a leaf node.

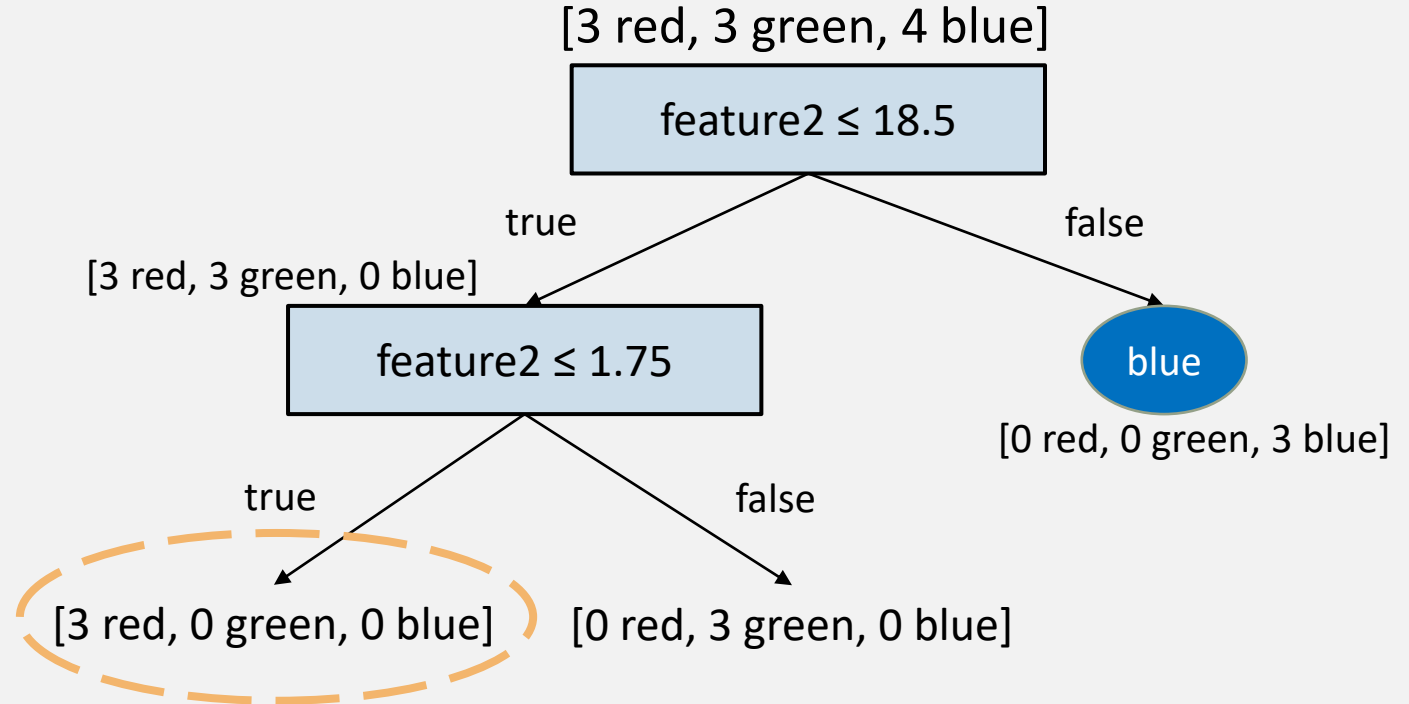
feature1	feature2	output
circle	-1.2	red
circle	-0.36	red
square	0.4	red
square	3.1	green
square	9	green
circle	15	green
circle	22	blue
circle	28	blue
square	32.1	blue
circle	39.8	blue



Decision Tree Building – Continuous features [8]

- Proceed to process next non-leaf node.

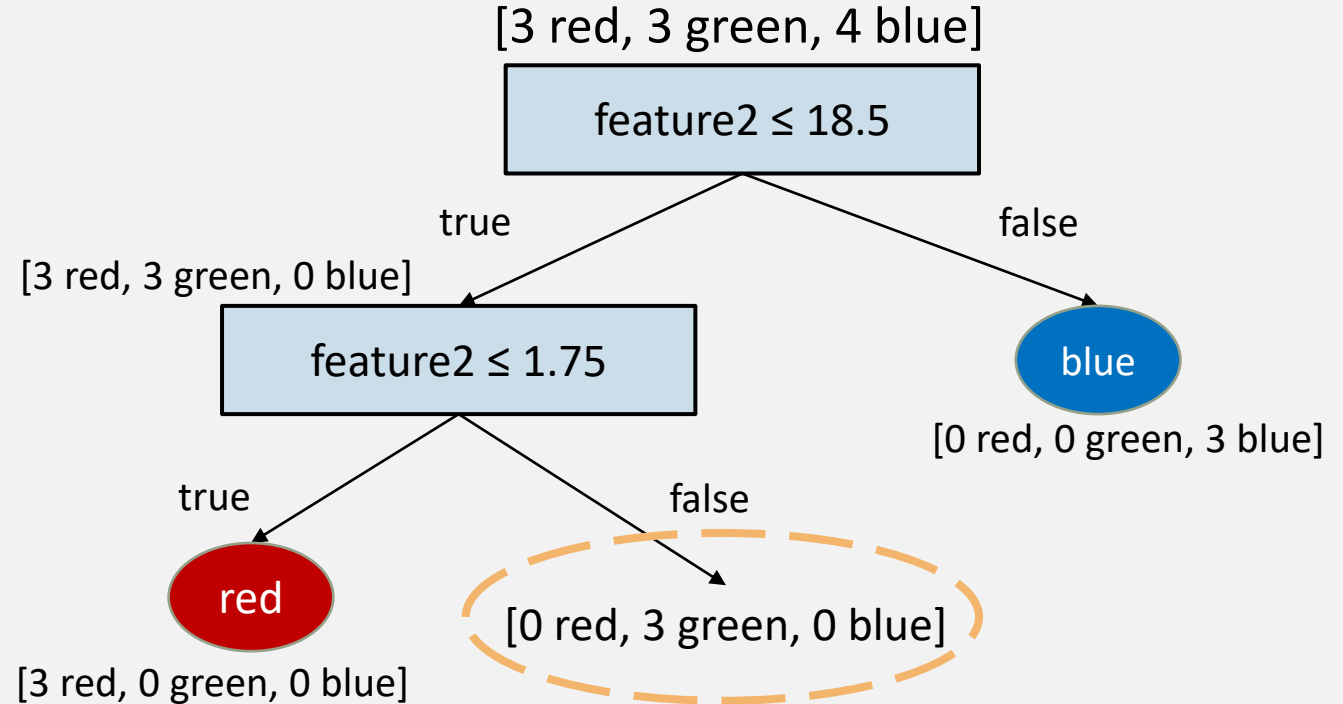
feature1	feature2	output
circle	-1.2	red
circle	-0.36	red
square	0.4	red
square	3.1	green
square	9	green
circle	15	green
circle	22	blue
circle	28	blue
square	32.1	blue
circle	39.8	blue



Decision Tree Building – Continuous features [9]

- Proceed to process next non-leaf node.

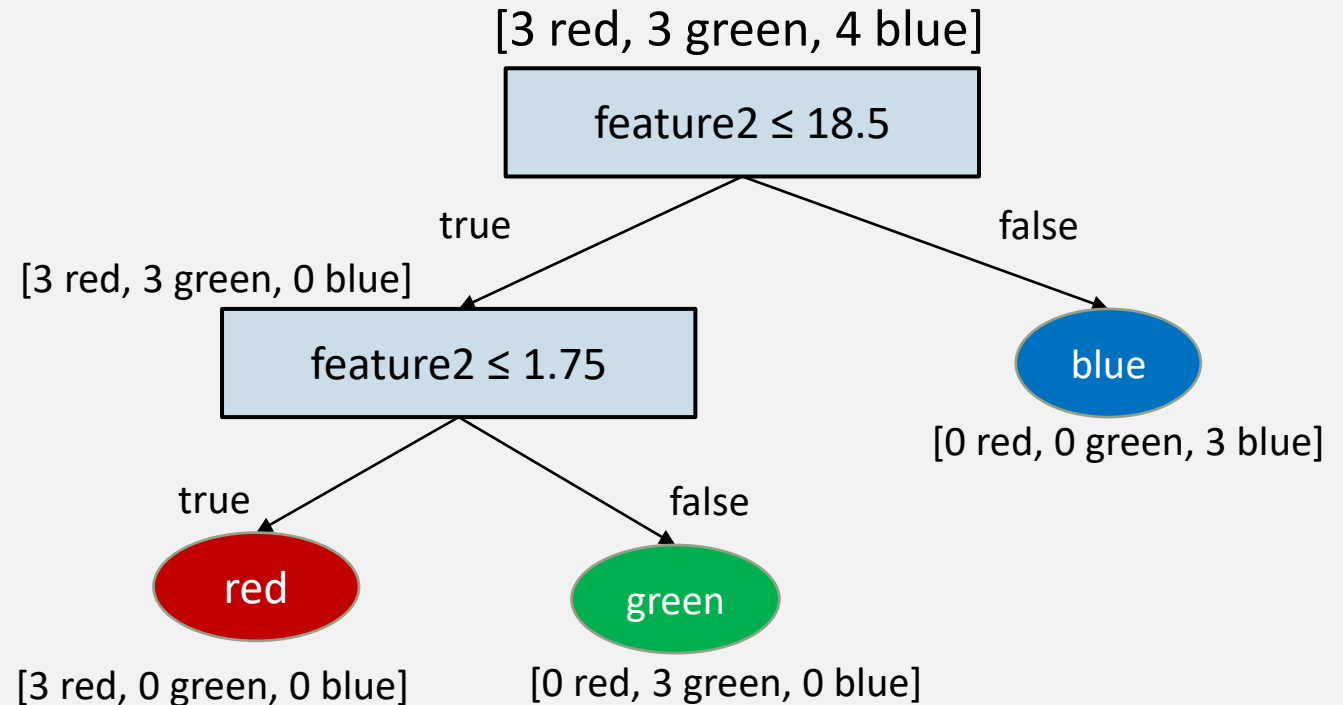
feature1	feature2	output
circle	-1.2	red
circle	-0.36	red
square	0.4	red
square	3.1	green
square	9	green
circle	15	green
circle	22	blue
circle	28	blue
square	32.1	blue
circle	39.8	blue



Decision Tree Building – Continuous features [9]

- Proceed to process next non-leaf node.
- We are done processing all non-leaf nodes.

feature1	feature2	output
circle	-1.2	red
circle	-0.36	red
square	0.4	red
square	3.1	green
square	9	green
circle	15	green
circle	22	blue
circle	28	blue
square	32.1	blue
circle	39.8	blue



Decision Tree Building – Continuous features [10]

- What is the tree prediction on $x_{new} = [circle, 20]$
- $y_{new} = blue$

